Consider a to-do list application again. You might find you have a Razor Page called ViewToDo.cshtml that displays a single to-do with a given `id`. Later, you create a new Razor Page, RecentToDos.cshtml, that displays the five most recent to-do items. Instead of copying and pasting the code from one page to the other, you could create a partial view, called _ToDo.cshtml, as in the following listing.

Listing 17.12 Partial view _ToDo.cshtml for displaying a `ToDoItemViewModel`

```
@model ToDoItemViewModel                ❶
<h2>@Model.Title</h2>                   ❷
<ul>                                    ❷
    @foreach (var task in Model.Tasks)  ❷
    {                                   ❷
        <li>@task</li>                  ❷
    }                                   ❷
</ul>                                   ❷
```

❶ Partial views can bind to data in the Model property, like a normal Razor Page uses a PageModel.

❷ The content of the partial view, which previously existed in the ViewToDo.cshtml file

Partial views are a bit like Razor Pages without the `PageModel` and handlers. Partial views are purely about rendering small sections of HTML rather than handling requests, model binding, and validation, and calling the application model. They are great for encapsulating small usable bits of HTML that you need to generate on multiple Razor Pages.

Both the ViewToDo.cshtml and RecentToDos.cshtml Razor Pages can render the _ToDo.cshtml partial view, which handles generating the HTML for a single class. Partial views are rendered using the `<partial />` Tag Helper, providing the name of the partial view to render and the data (the model) to render. For example, the RecentToDos .cshtml view could achieve this as shown in the following listing.